The Journal of Open Source Software

smolyax: a high-performance implementation of the Smolyak interpolation operator in JAX

Josephine Westermann [1¶] **and Joshua Chen** [2]

1 Heidelberg University, Germany **2** The University of Texas at Austin, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 15 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The `smolyax` library provides interpolation capabilities for arbitrary multivariate and vector-valued functions $f : \mathbb{R}^{d_1} \to \mathbb{R}^{d_2}$ for any $d_1, d_2 \in \mathbb{N}$.

It implements the Smolyak interpolation operator, which is known to overcome the curse-of-dimensionality plaguing naive multivariate interpolation (Barthelmann et al., 2000). The implementation is based on JAX (Bradbury et al., 2018), a free and open-source Python library for high-performance computing that integrates with the Python and NumPy numerical computing ecosystem. Thanks to JAX's device management, `smolyax` runs natively on both CPU and GPU. While implementing Smolyak interpolation in JAX is challenging due to the highly irregular data structures involved, `smolyax` overcomes this by employing a tailored batching and padding strategy, enabling efficient vectorization, scalability, and parallel execution.

`smolyax` supports interpolation on bounded or unbounded domains via Leja (M. A. Chkifa, 2013) or Gauss-Hermite (Abramowitz & Stegun, 1964) node sequences, respectively. It provides efficient Numba-accelerated routines to generate isotropic or anisotropic total degree multi-index sets (Adcock et al., 2022, sec. 2.3.2), which are the key ingredient to generate the high-dimensional sparse grids (Bungartz & Griebel, 2004) of interpolation nodes required by the Smolyak interpolation operator. Additional types of node sequences or multi-index sets can be incorporated easily by implementing a minimalistic interface.

Statement of Need

Polynomial expansion is a well-studied and powerful tool in applied mathematics, with important applications in surrogate modeling, uncertainty quantification and inverse problems, see e.g. (Adcock et al., 2022; A. Chkifa et al., 2015; Dũng et al., 2023; Herrmann et al., 2024; Westermann et al., 2025; Zech, 2018) and references therein.

Smolyak interpolation offers a practical way to construct polynomial approximations with known error bounds for a wide range of function classes, see e.g. (Barthelmann et al., 2000; A. Chkifa et al., 2015).

While several libraries provide high-dimensional interpolation functionality, none, to our knowledge, provides a hardware-agnostic, high performance implementation. `smolyax` addresses this gap by providing an efficient solution within the popular JAX ecosystem.

High-dimensional interpolation with the Smolyak operator

We briefly summarize the essentials of high-dimensional interpolation, where sparse grids have become the standard choice for interpolation points. In this setting, the interpolation operator

is commonly referred to as the Smolyak operator. This overview provides background and establishes notation, which will be used to describe our specific implementation choices in the next section. For ease of notation, the following discussion will focus on scalar-valued interpolation targets (i.e., on the case $d_1 = d$ and $d_2 = 1$), while the extension to vector-valued functions follows straightforwardly by interpolating each element of the output vector individually.

Univariate interpolation. Given a domain $D \subset \mathbb{R}$ and set of $\nu \in \mathbb{N}$ pairwise distinct interpolation points $(\xi_i^\nu)_{i=0}^\nu \subset D$, the polynomial interpolation operator $I^\nu : C^0(D) \to \mathbb{P}_\nu := \mathrm{span}\{x^i : i = 0, \dots, \nu\}$ maps a function $f : D \to \mathbb{R}$ onto the unique polynomial $I^\nu[f]$ of maximal degree ν such that $f(\xi_i^\nu) = I^\nu[f](\xi_i^\nu)$ for all $i \in \{0, 1, \dots, \nu\}$.

A classical method for evaluating the interpolating polynomial that is known for numerical stability is barycentric interpolation (Berrut & Trefethen, 2004). The univariate barycentric interpolation formula is given as

$$I^\nu[f](x) := \frac{\sum_{i=0}^\nu b_i^\nu(x) f(\xi_i^\nu)}{\sum_{i=0}^\nu b_i^\nu(x)}, \qquad \text{with } b_i^\nu(x) := \begin{cases} 1 & \text{if } i = \nu = 0 \\ \frac{w_i^\nu}{x - \xi_i^\nu} & \text{else} \end{cases} \tag{1}$$

$$\text{and } w_i^\nu := \prod_{k \in \{0,1,\dots,\nu\}/\{i\}} \frac{1}{(\xi_i^\nu - \xi_k^\nu)}. \tag{2}$$

Tensorized interpolation generalizes univariate interpolation to multivariate functions defined on a tensor-product domain $D = \otimes_{j=1}^d D_1$ with $D_1 \subset \mathbb{R}$ and $d \in \mathbb{N}$. Given a multi-index $\boldsymbol{\nu} \in \mathbb{N}_0^d$ characterizing the maximal polynomial degree in each dimension, we define $I^{\boldsymbol{\nu}} : C^0(D) \to \mathbb{P}_{\boldsymbol{\nu}} := \mathrm{span}\{\boldsymbol{x}^{\boldsymbol{\mu}} : \boldsymbol{\mu} \leq \boldsymbol{\nu}\}$ as

$$I^{\boldsymbol{\nu}} := \otimes_{j=1}^d I^{\nu_j}. \tag{3}$$

The tensorized barycentric interpolation formula can be expressed as a repeated vector-tensor product

$$I^{\boldsymbol{\nu}}[f](\boldsymbol{x}) = \frac{\boldsymbol{b}^{\nu_1}(x_1)\boldsymbol{b}^{\nu_2}(x_2)\cdots\boldsymbol{b}^{\nu_d}(x_d)\boldsymbol{F}^{\boldsymbol{\nu}}}{\prod_{j=1}^d (\sum_{i=0}^{\nu_j} b_i^{\nu_j}(x_j))}, \tag{4}$$

with tensors $\boldsymbol{F}^{\boldsymbol{\nu}} \in \mathbb{R}^{\boldsymbol{\nu}+\mathbf{1}} := \mathbb{R}^{(\nu_1+1)\times(\nu_2+1)\times\cdots\times(\nu_d+1)}$ given as

$$F_{\boldsymbol{\mu}}^{\boldsymbol{\nu}} := f(\boldsymbol{\xi}_{\boldsymbol{\mu}}^{\boldsymbol{\nu}}) \text{ with } \boldsymbol{\xi}_{\boldsymbol{\mu}}^{\boldsymbol{\nu}} := (\xi_{\mu_1}^{\nu_1}, \xi_{\mu_2}^{\nu_2}, \dots, \xi_{\mu_d}^{\nu_d}) \qquad \text{for all } \boldsymbol{\mu} \in \mathbb{N}_0^d \text{ s.t. } \boldsymbol{\mu} \leq \boldsymbol{\nu},$$

and vectors $\boldsymbol{b}^{\nu_j}(x_j) \in \mathbb{R}^{\nu_j+1}$ given as

$$\boldsymbol{b}^{\nu_j}(x_j) := (b_i^{\nu_j}(x_j))_{i=0}^{\nu_j} \qquad \text{for all } j \in \{1, \dots, d\}.$$

Smolyak interpolation (Barthelmann et al., 2000; Smolyak, 1963) overcomes the curse-of-dimensionality that characterizes tensor-product interpolation (4), by introducing polynomial ansatz spaces $\mathbb{P}_\Lambda := \mathrm{span}\{\boldsymbol{x}^{\boldsymbol{\mu}} : \boldsymbol{\mu} \in \Lambda\}$ parametrized by downward closed multi-index sets $\Lambda \subset \mathbb{N}_0^d$. The resulting interpolation operator is then a linear combination of tensorized interpolation operators:

$$I^\Lambda := \sum_{\boldsymbol{\nu} \in \Lambda} \zeta_{\Lambda,\boldsymbol{\nu}} I^{\boldsymbol{\nu}}, \qquad \zeta_{\Lambda,\boldsymbol{\nu}} := \sum_{\boldsymbol{e} \in \{0,1\}^d : \boldsymbol{\nu}+\boldsymbol{e} \in \Lambda} (-1)^{|\boldsymbol{e}|}. \tag{5}$$

A vectorizable implementation of the Smolyak operator

To leverage key HPC techniques such as vectorization, parallelization, and batch processing, input data must conform to a uniform structure. However, the vectors and tensors in (5)

DRAFT

together with (4) can exhibit a wide range of shapes, posing a challenge for efficient vectorization. A naive approach would be to zero-pad all tensors $\boldsymbol{F}^{\boldsymbol{\nu}}$ in (5) to the smallest possible common shape $(\max_{\boldsymbol{\nu}\in\Lambda}(\nu_j))_{j=1}^d$. This approach, however, suffers from the curse-of-dimensionality, as memory requirements grow exponentially with d. With our implementation we navigate in between these two extremes of handling a large number of small tensors and a single, massive tensor. The key idea is to set up all tensors by:

1. Dropping indices ("*squeezing*") of non-active dimensions, i.e., j with $\nu_j = 0$,
2. Permuting the remaining active dimensions in descending order, and
3. Zero-padding all tensors with the same number of active dimensions to the smallest common shape.

This reorganizes the tensors into a small number of large, structured data blocks, enabling efficient computation while keeping memory overhead modest.

Squeezing and permuting the dimensions of the tensorized interpolator. For any multi-index $\boldsymbol{\nu} \in \mathbb{N}_0^d$, denote with $s(\boldsymbol{\nu})$ the tuple consisting of the elements in $\{j \in \{1, ..., d\} \ : \ \nu_j > 0\}$ that permute the non-zero entries of $\boldsymbol{\nu}$ descendingly. Let further $\boldsymbol{\nu}^s := (\nu_j)_{j\in s(\boldsymbol{\nu})}$ and $d_{\boldsymbol{\nu}} := |\boldsymbol{\nu}_s| \equiv |s(\boldsymbol{\nu})| = |\{j \in \{1, ..., d\} \ : \ \nu_j > 0\}| \leq d$.

Example: For the multi-index $\boldsymbol{\nu} = (3, 0, 2, 0, 4) \in \mathbb{N}_0^5$, it holds that $s(\boldsymbol{\nu}) = (5, 1, 3)$, $\boldsymbol{\nu}^s = (4, 3, 2)$ and $d_{\boldsymbol{\nu}} = 3$.

With the above notation we can re-express the tensorized interpolation operator for $\boldsymbol{\nu}$ as

$$I^{\boldsymbol{\nu}}[f](\boldsymbol{x}) \equiv I^{\boldsymbol{\nu},s}[f](\boldsymbol{x}) := \frac{\boldsymbol{b}^{\nu_1^s}(x_{s_1(\boldsymbol{\nu})})\boldsymbol{b}^{\nu_2^s}(x_{s_2(\boldsymbol{\nu})}) \cdots \boldsymbol{b}^{\nu_{d_{\boldsymbol{\nu}}}^s}(x_{s_{d_{\boldsymbol{\nu}}}(\boldsymbol{\nu})})\boldsymbol{F}^{\boldsymbol{\nu},s}}{\prod_{j=1}^{d_{\boldsymbol{\nu}}}(\sum_{i=0}^{\nu_j^s} b_i^{\nu_j^s}(x_{s_j(\boldsymbol{\nu})}))} \quad \forall \boldsymbol{x} \in D, \tag{6}$$

with $\boldsymbol{F}^{\boldsymbol{\nu},s} \in \mathbb{R}^{\boldsymbol{\nu}^s+1}$ given as $F_{\boldsymbol{\mu}}^{\boldsymbol{\nu},s} := f(\boldsymbol{\xi}_{\boldsymbol{\mu}}^{\boldsymbol{\nu},s})$ for all $\boldsymbol{\mu} \in \mathbb{N}_0^{d_{\boldsymbol{\nu}}}$ s.t. $\boldsymbol{\mu} \leq \boldsymbol{\nu}^s$, with

$$\left(\boldsymbol{\xi}_{\boldsymbol{\mu}}^{\boldsymbol{\nu},s}\right)_j := \begin{cases} \xi_{\mu_i}^{\nu_j} & \text{if } j \in s(\boldsymbol{\nu}) \text{ and } j = s_i(\boldsymbol{\nu}) \\ \xi_0^{\nu_j} & \text{else.} \end{cases}$$

This construction is equivalent to (4) since for any dimension j with $\nu_j = 0$ it holds that the corresponding weight vector $\boldsymbol{b}^{\nu_j}(x_j)$ is a one-dimensional vector with entry 1, i.e. $\boldsymbol{b}^{\nu_j}(x_j) = (1) \in \mathbb{R}^1$.

Padding tensorized interpolators to a common shape. For any multi-index $\boldsymbol{\tau} \in \mathbb{N}_0^k$, define the padding operator $p^{\boldsymbol{\tau}}$ that acts on any order-k tensor $\boldsymbol{T} \in \mathbb{R}^{\boldsymbol{\rho}}$ with $\boldsymbol{\rho} \leq \boldsymbol{\tau}$ as

$$p^{\boldsymbol{\tau}}(\boldsymbol{T}) \in \mathbb{R}^{\boldsymbol{\tau}}, \ p^{\boldsymbol{\tau}}(\boldsymbol{T})_{\boldsymbol{\mu}} := \begin{cases} T_{\boldsymbol{\mu}} & \text{if } \boldsymbol{\mu} \leq \boldsymbol{\rho} \\ 0 & \text{else.} \end{cases} \quad \forall \boldsymbol{\mu} \leq \boldsymbol{\tau}.$$

Example: For $\boldsymbol{\rho} = (2, 3)$, the tensor $\boldsymbol{T} = \begin{pmatrix} 0 & 3 & -1 \\ 2 & 0 & 4 \end{pmatrix} \in \mathbb{R}^{\boldsymbol{\nu}}$ can be padded to $\boldsymbol{\tau} = (3, 5)$ as

$$p^{\boldsymbol{\tau}}(\boldsymbol{T}) = \begin{pmatrix} 0 & 3 & -1 & 0 & 0 \\ 2 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \in \mathbb{R}^{\boldsymbol{\tau}}.$$

Any tensorized interpolation operator $I^{\boldsymbol{\nu}}$ can be padded to higher dimensions $\boldsymbol{\tau} \geq \boldsymbol{\nu}$ via

$$I^{\boldsymbol{\nu}}[f](\boldsymbol{x}) \equiv I^{\boldsymbol{\nu},\boldsymbol{\tau}}[f](\boldsymbol{x}) := \frac{p^{\tau_1}(\boldsymbol{b}^{\nu_1}(x_1))p^{\tau_2}(\boldsymbol{b}^{\nu_2}(x_2)) \cdots p^{\tau_d}(\boldsymbol{b}^{\nu_d}(x_d))p^{\boldsymbol{\tau}}(\boldsymbol{F}^{\boldsymbol{\nu}})}{\prod_{j=1}^d(\sum_{i=0}^{\tau_j} p_i^{\tau_j}(\boldsymbol{b}^{\nu_j}(x_j)))}. \tag{7}$$

DRAFT

We write $I^{\boldsymbol{\nu},s,\boldsymbol{\tau}}$ when applying (6) and (7) successively.

Pseudocode of our implementation strategy. We now have everything in place to construct the Smolyak interpolant in a form that is well-suited for vectorized execution. Algorithm 1 outlines the key steps. Given a downward-closed but otherwise arbitrarily structured multi-index set Λ and a target function f, we begin by identifying the subset Λ_ζ of multi-indices $\boldsymbol{\nu}$ with nonzero Smolyak coefficients $\zeta_{\Lambda,\boldsymbol{\nu}}$ and determining the maximal number $N \leq d$ of nonzero entries across these multi-indices. Notably, N remains small (typically single-digit) even when the dimensionality d reaches the hundreds and $|\Lambda|$ is on the order of tens of thousands. For each fixed sparsity level $n \in [0, \dots, N]$, we extract the subset Λ_n of multi-indices with exactly n nonzero entries and determine the minimal bounding multi-index $\boldsymbol{\tau} \in \mathbb{N}_0^n$ such that $\boldsymbol{\nu}^s \leq \boldsymbol{\tau}$ for all $\boldsymbol{\nu} \in \Lambda_n$. This step ensures that all the tensorized interpolation operators $(I^{\boldsymbol{\nu}})_{\boldsymbol{\nu}\in\Lambda_n}$ can be efficiently assembled into a single, vectorized computation. In Algorithm 1, this is compactly expressed as the summation of all $(I^{\boldsymbol{\nu},s,\boldsymbol{\tau}})_{\boldsymbol{\nu}\in\Lambda_n}$, but in practice, it corresponds to pre-allocating and incrementally populating large arrays for interpolation nodes, weights, and function values. The final interpolant I^Λ is then assembled through a brief loop over a small number of high-throughput operations, ensuring computational efficiency.

Algorithm 1 Construct the barycentric Smolyak interpolator I^Λ
Input: Target function f, multi-index set $\Lambda \subset \mathbb{N}_0^d$
Output: I^Λ

$I^\Lambda = 0$
$\Lambda_\zeta = \{\boldsymbol{\nu} \in \Lambda \,:\, \zeta_{\Lambda,\boldsymbol{\nu}} \neq 0\}$
$N = \max\{d_{\boldsymbol{\nu}} \,:\, \boldsymbol{\nu} \in \Lambda_\zeta\}$
for $n \in \{0, \dots, N\}$ **do**
  $\Lambda_n = (\boldsymbol{\nu} : \boldsymbol{\nu} \in \Lambda_\zeta,\ d_{\boldsymbol{\nu}} = n)$
  $\boldsymbol{\tau} = (\max(\{\nu_i^s \,:\, \boldsymbol{\nu} \in \Lambda_n\}))_{i=1}^n$
  $I^{\Lambda,n} = 0$
  for $\boldsymbol{\nu} \in \Lambda_n$ **do**
    $I^{\Lambda,n} += \zeta_{\Lambda,\boldsymbol{\nu}} I^{\boldsymbol{\nu},s,\boldsymbol{\tau}}$
  end for
  $I^\Lambda += I^{\Lambda,n}$
end for
return I^Λ

Acknowledgements

We thank Thomas O'Leary-Roseberry and Jakob Zech for insightful discussions in the early stages of this project.

References

Abramowitz, M., & Stegun, I. A. (1964). *Handbook of mathematical functions with formulas, graphs, and mathematical tables: Vols. No. 55* (p. xiv+1046). U. S. Government Printing Office, Washington, DC.

Adcock, B., Brugiapaglia, S., & Webster, C. G. (2022). *Sparse polynomial approximation of high-dimensional functions* (Vol. 25, p. xvii+292). Society for Industrial; Applied Mathematics (SIAM), Philadelphia, PA. https://doi.org/10.1137/1.9781611976885

Barthelmann, V., Novak, E., & Ritter, K. (2000). High dimensional polynomial interpolation on sparse grids. In *Adv. Comput. Math.* (Vol. 12, pp. 273–288). https://doi.org/10.1023/A:1018977404843

DRAFT

Berrut, J.-P., & Trefethen, L. N. (2004). Barycentric Lagrange interpolation. *SIAM Rev.*, *46*(3). https://doi.org/10.1137/S0036144502417715

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.5.1). http://github.com/jax-ml/jax

Bungartz, H.-J., & Griebel, M. (2004). Sparse grids. *Acta Numer.*, *13*, 147–269. https://doi.org/10.1017/S0962492904000182

Chkifa, A., Cohen, A., & Schwab, C. (2015). Breaking the curse of dimensionality in sparse polynomial approximation of parametric PDEs. *J. Math. Pures Appl. (9)*, *103*(2), 400–428. https://doi.org/10.1016/j.matpur.2014.04.009

Chkifa, M. A. (2013). On the Lebesgue constant of Leja sequences for the complex unit disk and of their real projection. *J. Approx. Theory*, *166*, 176–200. https://doi.org/10.1016/j.jat.2012.11.005

Dũng, D., Nguyen, V. K., Schwab, C., & Zech, J. (2023). *Analyticity and sparsity in uncertainty quantification for PDEs with Gaussian random field inputs* (Vol. 2334, p. xv+205). Springer, Cham. https://doi.org/10.1007/978-3-031-38384-7

Herrmann, L., Schwab, C., & Zech, J. (2024). Neural and spectral operator surrogates: Unified construction and expression rate bounds. *Adv. Comput. Math.*, *50*(4), Paper No. 72, 43. https://doi.org/10.1007/s10444-024-10171-2

Smolyak, S. A. (1963). Quadrature and interpolation formulas for tensor products of certain classes of functions. *Doklady Akademii Nauk*, *148*, 1042–1045.

Westermann, J., Huber, B., O'Leary-Roseberry, T., & Zech, J. (2025). *Performance of neural and polynomial operator surrogates*.

Zech, J. (2018). *Sparse-Grid Approximation of High-Dimensional Parametric PDEs*. http://dx.doi.org/10.3929/ethz-b-000340651

DRAFT